# LabPulse: Smart Equipment Fault Reporting & Duplicate Reduction System

**Internship Report | ShadowFox Cloud Engineer (AWS) | Program ID SF-PID-2026-W7290**

Adithyan P S, Integrated MCA, Amrita Vishwa Vidyapeetham, Kochi | October 1 to 31, 2026

## 1. Overview

During this internship I designed, built and tested LabPulse, a serverless system on AWS for reporting lab equipment faults. A QR code on each machine opens a report form with the equipment ID filled in. The page first shows whether that machine already has an open report. If it does, a new report is merged into it and a counter rises, so repeated faults become a visible priority signal for lab staff.

The work covers the Beginner and Intermediate levels of the task list. The Advanced level was not completed, and this report does not claim it.

## 2. Architecture and service choices

| Service | Role and reason |
|---|---|
| Amazon S3 | Static website hosting for the report form and the staff dashboard. Low cost and no servers |
| API Gateway (HTTP API) | Public HTTPS endpoints with four routes, CORS limited to one origin |
| AWS Lambda (Python) | One function that routes requests, validates input and applies the merge rule |
| Amazon DynamoDB | Stores reports. Partition key equipment ID, sort key report ID, so one query returns all reports for a machine |
| CloudWatch Logs | Records each saved report and each status change |

I chose DynamoDB over RDS because the access pattern is simple, it needs no VPC or server, and it costs little at lab scale. Region: Asia Pacific (Mumbai).

## 3. Work completed

**Beginner.** A static form hosted on S3, a Lambda function behind API Gateway, validation in the browser and again on the server, and clear success and failure messages.

**Intermediate.** Reports are stored in DynamoDB. A duplicate report on a machine with an open report increments its count through a conditional update. A public status route returns only the status and count. Two staff-only routes list reports and change status between open, in progress and resolved. A staff dashboard shows counts, filters and status controls, and marks machines reported three or more times.

## 4. Security and cost

- The Lambda role allows four DynamoDB actions on this one table only.
- Staff routes need a key held in a Lambda environment variable, compared in constant time. Without it the API returns 403.
- CORS allows one origin. The bucket policy allows only reading site files.
- The account is on the AWS Free plan with credits and a zero-spend budget alert. The DynamoDB table uses 1 read and 1 write unit with auto scaling off.
- The reporter's email is stored but never returned by a public route.

## 5. Testing

I tested valid and invalid input in the Lambda console, over HTTP with PowerShell, and in the browser. A PowerShell script in the repository checks that staff routes return 403 without the key and 200 with it. DynamoDB showed one item with a count of two after two reports on the same machine. CloudWatch logs confirmed each request reached Lambda.

## 6. Problems solved

- Lambda tests returned the default text because I had not clicked Deploy. I now deploy before every test.
- The first DynamoDB table was created without a sort key, which cannot be changed later, so I deleted and recreated it.
- The form failed with a CORS error because the browser blocked an empty allowed-headers setting, and later because I opened the page from the S3 object address instead of the website address. I fixed the header and kept a single allowed origin.

## 7. Limitations and next steps

The site is served over HTTP, the staff key is a shared secret rather than per-user login, API rate limiting was not verified so it is not claimed, and listing reports uses a Scan. Next steps are CloudFront for HTTPS, Cognito for staff login, CloudWatch alarms and a repeatable deployment template.

## 8. What I learned

I learned how S3, API Gateway, Lambda and DynamoDB fit together, why server-side validation matters, how CORS compares origins exactly, and how to plan access, cost and cleanup before building.

**Links:** GitHub: https://github.com/Adithyanps47/labpulse | Demo video: in the repository and on Google Drive.